

# ARCHITECTURAL DESIGN REVIEW & RISK ASSESSMENT

---

**Document Title:** SQS FIFO Pipeline Redesign: Comparative Analysis of Event-Splitting vs. Tenant Batching

**Author:** Shripad Algude, DevOps & Core Platform Engineering

**Date:** June 15, 2026

---

## 1. Executive Summary & Strategic Verdict

This assessment provides a comparative technical audit of the two proposed architectural paths for the ACIS SQS FIFO queue migration:

- **1. The Event-Splitting Path:**  
Mutating multi-environment ARM event payloads into isolated single-environment messages, using `currentDeployEnvId` as the SQS `MessageGroupId`.
- **2. The Tenant Batching Path (Recommended):**  
Preserving multi-environment payloads intact, using the customer's Tenant ID (`ECA_ID`) as the SQS `MessageGroupId`, combined with an upstream Batch Size of 5 and AWS `ReportBatchItemFailures`.

### Strategic Verdict

**The Event-Splitting approach is technically invalid and should not be implemented.**

While target Teamcenter deployments execute on isolated endpoints with separate URLs and configurations, the orchestration framework inside ACIS handles global, stateful tenant lifecycle transitions. Dividing the payload introduces fatal distributed race conditions, multi-tenant compute starvation, and silent data erasure.

Implementing the splitting design would require a high-risk refactoring of dozens of foundational state files. Conversely, the Tenant Batching (ECA-ID) approach resolves our performance goals completely at the infrastructure tier, requiring zero changes to our core business state logic.

**From my perspective, the ECA-ID approach is vastly superior, highly effective, and completely maintains data integrity. I request authorization to take full ownership of implementing the ECA-ID solution.**

---

## 2. Code-Level Evidence: Why the Splitting Logic Fails

### Failure 1: Asymmetric State Evaluation & Rogue User Deactivations

- **File Location:** events/user\_provisioning\_event.py

```
previous_deploy_env_ids = list(set(event_data.previous_deploy_env_ids) -  
set(event_data.current_deploy_env_ids))  
for env_id in previous_deploy_env_ids:  
    context_data.current_deploy_env_id = env_id  
    context_data.is_previous_deploy_env_unprovision_event = True  
    super().handle(..., initial_state=UpsTenantMetadataFetchingState)
```

#### Failure Mechanism:

The framework calculates environmental offboarding deltas using mathematical set subtraction. If an administrator modifies an entitlement so a user remains active in [Dev, UAT] but is removed from [Prod], the splitting approach divides this transaction into parallel execution threads.

Lambda Instance 1 (handling Dev) receives a payload where current\_deploy\_env\_ids is isolated to only [Dev], while previous\_deploy\_env\_ids retains the complete history of [Dev, UAT, Prod]. When Lambda 1 executes the subtraction, the math resolves to: [Dev, UAT, Prod] - [Dev] = [UAT, Prod]. It then loops through UAT and explicitly invokes UpsTcxUserDeactivationState. Concurrently, Lambda Instance 2 (handling UAT) resolves its math to [Dev, Prod] and deactivates the user from Dev.

**Business Impact:** Critical Production Bug. Parallel executors will cross-fire, silently erasing active user access across valid corporate environments.

### Failure 2: Breaking Web Gateway Lifecycle Contracts

- **File Location:** events/user\_provisioning\_event.py

```
# Send notifications and acknowledge the event after all deploy-env processing  
super().handle(  
    event_data=event_data,  
    context_data=context_data,  
    state_transitions=state_transitions,  
    initial_state=PostArmEventNotificationState,  
)
```

#### Failure Mechanism:

The upstream ARM platform adheres to a tight relational transaction schema. It mandates exactly one terminal status acknowledgment webhook execution per unique transactional requestId.

By dividing a single event into multiple messages, separate Lambda instances arrive at this trailing orchestration block independently. The fastest thread successfully updates ARM. When parallel threads 2 and 3 execute the exact same state, the upstream ARM gateway API will reject the call, throwing an HTTP 400 Bad Request or HTTP 409 Conflict because the parent transaction has already been closed.

**Business Impact:** The worker Lambda treats this gateway rejection as a fatal network drop, crashing the runtime execution loop and dumping valid user creation requests directly into the Dead Letter Queue (DLQ).

## Failure 3: The Immutable Telemetry Matrix Blocker

- **File Location:** events/event\_data/user\_provisioning\_event\_data.py

```
@classmethod
def from_dict(cls, data: Dict[str, Any]) -> "UserProvisioningEventData":
    return cls(
        ...
        correlation_id=data["requestId"],
```

### Failure Mechanism:

Framework telemetry, auditing, and duplicate-prevention locks are fundamentally bound to requestId. Attempting to bypass parallel database collisions by mutating this key (e.g., appending suffixes like req-123\_Dev) destroys the system's global audit trail.

Siemens monitoring tools track infrastructure operations under req-123, but Teamcenter audit logs will record updates under req-123\_Dev. This breaks cross-app debugging and compliance reconciliation. If we do not mutate the ID, parallel threads will simultaneously try to write to identical rows in tracking tables, triggering database Unique Constraint violations.

## Failure 4: Destruction of the Sequential Framework Defect LCS-1019883 Workaround

- **File Location:** states/ups\_user\_creation\_requested.py

```
is_first_user = self.request.context_data.is_first_user
if is_first_user:
    tms_client = TenantMetadataAccessor.get_tms_client(...)
    response = tms_client.create_essentials_tenant_deployment_entry(...)

make_user_and_tenant_soa.invoke(is_tenant_creation=is_first_user)
if is_first_user:
    # Defect: LCS-1019883 - User not created when onboard first user...
    make_user_and_tenant_soa.invoke(is_tenant_creation=False)
```

### Failure Mechanism:

For Essentials tiers, the framework implements a delicate, two-step consecutive API handshake explicitly written to bypass a known Siemens Core Server Bug (Defect LCS-1019883).

If multi-environment events are split and run in parallel, multiple Lambda instances will execute `is_first_user = True` simultaneously before any data can be saved to the database. Both threads will issue duplicate registration POST calls to `create_essentials_tenant_deployment_entry`, throwing HTTP 409 database conflicts. Furthermore, firing overlapping initialization handshakes shatters the strict sequential timing required to satisfy the server-side workaround, breaking environment creation.

## Failure 5: Re-Entrant Memory Accumulation Corruption

- **File Location:** events/event\_data/user\_provisioning\_event\_data.py

```
@dataclass
class ContextData(BaseContextData):
    ...
    email_context_list: list = field(default_factory=list)
    notification_results: Dict[str, Any] = field(default_factory=dict)
    site_id_context_list: list = field(default_factory=list)
```

### Failure Mechanism:

The state machine framework relies on data aggregation. As the system sequentially checks environments, it logs metadata tokens and results directly into these memory list buffers. At the very end of processing, PostArmEventNotificationState reads these compiled lists to send one clean execution status report back to Siemens.

Splitting the payload forces separate Lambda instances to boot up with blank, isolated memory contexts. Worker 1 only has visibility into Dev; Worker 2 only has visibility into Production. The framework can no longer compile a unified processing state, blinding our automated reporting layers.

## Failure 6: Distributed Race Conditions on Centralized Target APIs

- **File Location:** states/ups\_update\_application\_info.py

```
request_body = {
    "ecaId": event_data.eca_id,
    "appUrls": [{"name": event_data.current_product, "link": tcx_url}],
    ...
}
sws_bas_client.update_application_info(request_body=request_body)
```

### Failure Mechanism:

This component invokes an external global Siemens Software Center API, using the customer's shared company ecaId as the unique database index key. Notice that the request body builds a single-item array containing only the link of the current execution thread.

If split environment updates run in parallel, they will concurrently hit this endpoint for the same corporate customer entity. The execution thread that resolves last will commit its single URL entry, completely overwriting and wiping out the links committed by the preceding thread, resulting in corrupted application directories in the Software Center application index.

## Failure 7: Socket Exhaustion via Unpooled Client Instantiation

- **File Location:** utils/api\_client.py

```
class ApiClient:
    def __init__(self, headers: Optional[Dict[str, str]] = None):
```

```
...
self._session = requests.Session()
```

#### Failure Mechanism:

The core networking utility implements `requests.Session()` to establish a persistent connection pool, safely reusing open TCP sockets sequentially across the environment processing sequence.

Splitting forces separate Lambda processes to spin up independent pools. Instead of reusing sockets efficiently, the architecture floods HashiCorp Vault and Teamcenter endpoints with overlapping connection handshakes. This triggers socket exhaustion and intermittent connection rejections at our target security gateways.

---

## 3. Infrastructure Cap & Multi-Tenant Concurrency Starvation

**System Constraint:** **Maximum System-Wide Lambda Concurrency Capacity = 5 Threads**

The Event-Splitting proposal presents an immediate infrastructure bottleneck: it allows a single "noisy neighbor" client to hijack our entire compute allocation.

Under SQS FIFO rules, parallel message processing is triggered across separate `MessageGroupIds`. If `currentDeployEnvId` is designated as the partition key, distinct environments under a single customer will execute simultaneously on separate Lambda instances.

### The Starvation Scenario

- **Step 1:** Customer A (e.g., BMW) triggers an administrative modification that splits into 5 single-environment messages (Dev, Test, Stage, Prod, Sandbox).
- **Step 2:** SQS FIFO identifies 5 separate `MessageGroupIds` and immediately boots up 5 parallel Lambda instances to process them at the exact same millisecond.
- **Step 3:** Customer A now consumes 5 out of 5 (100%) of our entire system's maximum Lambda concurrency capacity.
- **Step 4:** Customer B (e.g., Mercedes) drops an urgent user provisioning message into the queue at that exact same instance.

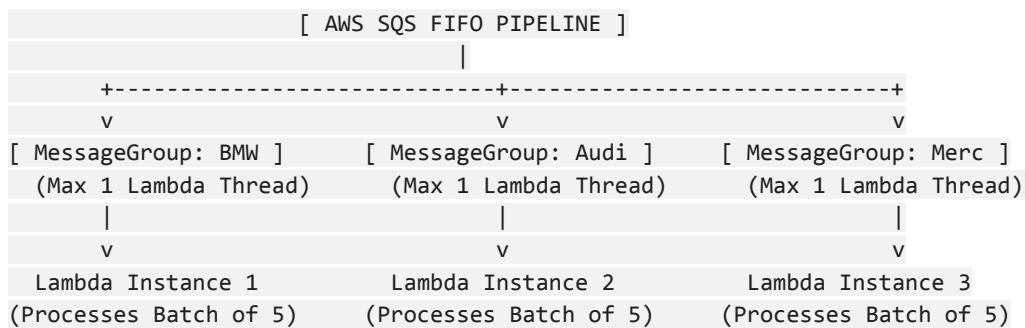
**Result:** Customer B's event is completely starved and blocked in the queue because Customer A has monopolized the entire compute pool. This breaks multi-tenancy SLAs.

---

# 5. The Recommended Compliant Path: Inter-Tenant Batching

To achieve optimal throughput while guaranteeing absolute data integrity and system safety, the system must utilize native AWS SQS FIFO batch features rather than application-level data mutations:

- 1. Map SQS MessageGroupId = ECA\_ID (Tenant ID)
- 2. Configure SQS Lambda Trigger Source Batch Size = 5
- 3. Enable Report Batch Item Failures in Lambda Trigger Settings



## Why This Approach Is Completely Superior

- **Guaranteed Multi-Tenant Fairness:** Grouping by ECA\_ID forces SQS FIFO to restrict processing to exactly one active Lambda instance per customer tenant at any given moment. Customer A can only ever occupy 1 Lambda slot, leaving the remaining 4 slots completely open to process Customer B, C, D, and E in parallel.
  - **100% Application Code Safety:** The multi-environment array remains fully unified within the payload. The system iterates through deployments sequentially inside a single thread, ensuring delta calculations (previous - current) and framework workaround sequences (Defect LCS-1019883) execute flawlessly out-of-the-box.
  - **Zero Refactoring Overhead:** Zero structural changes are required for core business state handlers, state registries, or down-stream tracking data schemas. We only update the core lambda\_handler entry loop to support standard batch array iteration.
  - **Socket-Efficient Connection Reuse:** Maximizes the performance of our requests.Session() connection pool inside ApiClient, maintaining a tight, clean socket footprint.
  - **Granular Fault Isolation:** By returning batchItemFailures, if message #2 in a batch of 5 fails due to a network blip, only that specific message drops back into the queue. The remaining 4 successful messages are cleanly deleted from the SQS infrastructure, preventing pipeline blocking without creating data-level bugs.
-

## 6. Engineering Action & Ownership Request

Based on this code-level root-cause analysis, the Event-Splitting approach poses an unacceptable risk to data integrity and framework stability. The Inter-Tenant Batching (ECA-ID) solution satisfies our performance scalability requirements cleanly at the infrastructure tier with zero application-level risks.

**I request formal authorization to bypass the event-splitting design and proceed directly with implementing the ECA-ID batching architecture via Terraform and lambda\_function loop updates.**

***I am fully prepared to take complete ownership of this rollout.***